



Implémentation du module ELK

Elasticsearch · Logstash · Kibana

Documentation technique · Code · Questions d'évaluation

Projet	Module	Points	Équipe
SafeSchool — ft_transcendence	DevOps — Gestion des logs ELK	2 points majeurs	mobougri · quclaque · eguthman · mdoan

1. C'est quoi ELK et pourquoi on l'a mis dans SafeSchool

1.1 Les 3 composants de la stack ELK

ELK est un acronyme pour trois outils qui fonctionnent ensemble comme une chaîne de traitement des logs. Dans SafeSchool, chaque action (connexion, signalement, calcul de score...) génère un log qui transite par cette chaîne.

■ NestJS (produit les logs) → Logstash (collecte + transforme) → Elasticsearch (stocke + indexe) → Kibana (visualise)

Composant	Rôle	Port	Analogie simple
Elasticsearch	Base de données des logs — stocke et indexe tout pour permettre des recherches ultra-rapides	9200	Une bibliothèque avec un index parfait
Logstash	Collecteur — reçoit les logs de NestJS, les enrichit (ajoute des tags, parse les dates) et les envoie vers Elasticsearch	5044 (TCP)	Un facteur qui trie le courrier
Kibana	Interface web — affiche des graphiques, tableaux de bord et permet de chercher dans les logs en temps réel	5601	Un tableau de bord Excel mais beau et interactif

1.2 Pourquoi c'est utile spécifiquement pour SafeSchool

Utilité 1 — Sécurité des données sensibles

SafeSchool contient des informations très sensibles sur des mineurs. Grâce aux logs d'authentification qu'on envoie vers Kibana, on peut détecter des tentatives d'intrusion : si quelqu'un essaie 10 fois un mauvais mot de passe sur le compte admin en 1 minute, c'est visible immédiatement dans Kibana.

Utilité 2 — Ne pas rater les signalements urgents

Chaque fois que l'IA détecte une urgence (menace physique, idées suicidaires), le log `scoring_event` arrive dans Elasticsearch avec le tag `urgency:true`. On peut configurer une alerte Kibana qui notifie l'admin en temps réel, avant même qu'il ouvre l'application.

Utilité 3 — Historique permanent de 30 jours

Les logs Docker disparaissent au redémarrage. Avec Elasticsearch, on conserve 30 jours d'historique complet. On peut répondre à des questions comme : qui a modifié le dossier #2026-001 ? À quelle heure ? L'élève a-t-il bien reçu la convocation ?

Utilité 4 — Statistiques invisibles dans l'application

Dans Kibana on peut voir : à quelle heure les élèves signalent le plus, quel type de harcèlement est le plus fréquent, combien de fois Groq a détecté une urgence ce mois-ci, quel est le temps de réponse moyen de l'API.

2. Architecture — Comment ça fonctionne dans le projet

2.1 Flux complet d'un log

Étape	Composant	Ce qui se passe
1	NestJS (LoggerService)	Un événement se produit (connexion, signalement...). LoggerService construit un objet JSON structuré.
2	LogstashTransport (TCP)	Le JSON est envoyé via TCP au port 5044 de Logstash. Si Logstash n'est pas dispo, le message est mis en tampon.
3	Logstash (pipeline)	Logstash reçoit le JSON, l'enrichit (ajoute des tags, parse le timestamp), et le transmet.
4	Elasticsearch	Le log est stocké dans l'index safeschool-logs-2026.04.15 et immédiatement indexé pour la recherche.
5	Kibana	L'admin ouvre Kibana et voit le log apparaître en temps réel dans les dashboards.

2.2 Les types de logs envoyés

Type de log	Champs envoyés	Exemple concret
http_request	method, url, statusCode, responseTime, userId, userRole, ip	POST /reports → 201 (245ms) — userId: abc123 — role: student
auth_event	action, email, userId, userRole, reason	login_failure — eleve@safeschool.com — reason: wrong_password
report_event	action, reportId, caseNumber, grade, score, status, userId	created — #2026-009 — grade: critique — score: 72
scoring_event	typeScore, freqScore, classScore, recidiveScore, roleScore, aiScore, finalScore, grade, urgency	type:25 freq:20 classe:7 → total:72 — critique — urgence:true

3. Fichiers créés et modifiés — avec le code

3.1 docker-compose.yml — Ajout des 4 nouveaux services ELK

On ajoute 4 services dans docker-compose.yml. Le service elk-setup est un conteneur spécial qui tourne une seule fois au démarrage pour créer la politique de rétention dans Elasticsearch.

docker-compose.yml

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
  container_name: safeschool_elasticsearch
  environment:
    - discovery.type=single-node # Mode mono-noeud (pas de cluster)
    - xpack.security.enabled=false # Sécurité désactivée (dev)
    - ES_JAVA_OPTS=-Xms512m -Xmx512m # Limite RAM JVM
  ports:
    - '9200:9200' # API REST Elasticsearch
  volumes:
    - esdata:/usr/share/elasticsearch/data # Persistance
  healthcheck:
    test: ['CMD-SHELL', 'curl -f http://localhost:9200/_cluster/health || exit 1']
    interval: 15s
    retries: 10

logstash:
  image: docker.elastic.co/logstash/logstash:8.11.0
  container_name: safeschool_logstash
  ports:
    - '5044:5044' # NestJS envoie ses logs ici via TCP
  volumes:
    - ./elk/logstash/pipeline:/usr/share/logstash/pipeline
    - ./elk/logstash/config/logstash.yml:/usr/share/logstash/config/logstash.yml
  depends_on:
    elasticsearch:
      condition: service_healthy

kibana:
  image: docker.elastic.co/kibana/kibana:8.11.0
  ports:
    - '5601:5601' # Interface web Kibana
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
```

```
elk-setup:
image: curlimages/curl:latest
volumes:
- ./elk/setup:/setup
depends_on:
elasticsearch:
condition: service_healthy
command: sh /setup/setup.sh # Tourne une seule fois
```

3.2 elk/logstash/pipeline/logstash.conf — Pipeline de traitement

Ce fichier définit comment Logstash traite les logs. Il a 3 sections : input (d'où viennent les logs), filter (comment on les transforme) et output (où on les envoie).

```
elk/logstash/pipeline/logstash.conf

input {
  tcp {
    port => 5044 # Ecoute les connexions TCP de NestJS
    codec => json_lines # Chaque ligne = un objet JSON
  }
}

filter {
  # Met le niveau de log en MAJUSCULES (INFO, ERROR, WARN)
  if [level] { mutate { uppercase => ['level'] } }

  # Parse le timestamp envoye par NestJS comme timestamp Elasticsearch
  if [timestamp] {
    date {
      match => ['timestamp', 'ISO8601']
      target => '@timestamp'
      remove_field => ['timestamp']
    }
  }

  # Ajoute des tags selon le type pour filtrer facilement dans Kibana
  if [type] == 'http_request' { mutate { add_tag => ['http'] } }
  if [type] == 'auth_event' { mutate { add_tag => ['auth'] } }
  if [type] == 'report_event' { mutate { add_tag => ['report'] } }
  if [level] == 'ERROR' { mutate { add_tag => ['error'] } }
}

output {
  elasticsearch {
    hosts => ['http://elasticsearch:9200']
```

```
index => 'safeschool-logs-%{+YYYY.MM.dd}' # Un index par jour
}

stdout { codec => rubydebug } # Aussi dans la console (debug)
}
```

3.3 elk/setup/ilm-policy.json + setup.sh — Rétention 30 jours

La politique ILM (Index Lifecycle Management) gère automatiquement le cycle de vie des logs : hot (actif, 7 jours), warm (archive, 23 jours), puis suppression. Le script setup.sh envoie cette politique à Elasticsearch au démarrage via curl.

ilm-policy.json + setup.sh

```
# elk/setup/ilm-policy.json
{
  "policy": {
    "phases": {
      "hot": { "actions": { "rollover": { "max_age": "7d" } } },
      "warm": { "min_age": "7d", "actions": { "forcemerge": { "max_num_segments": 1 } } },
      "delete": { "min_age": "30d", "actions": { "delete": {} } }
    }
  }
}

# elk/setup/setup.sh
until curl -s http://elasticsearch:9200/_cluster/health > /dev/null; do
  sleep 2 # Attend qu'Elasticsearch soit pret
done

curl -X PUT http://elasticsearch:9200/_ilm/policy/safeschool-logs-policy \
-H 'Content-Type: application/json' \
-d @/setup/ilm-policy.json
```

3.4 backend/src/logger/logstash.transport.ts — Connexion TCP vers Logstash

Ce fichier crée un transport Winston personnalisé qui envoie chaque log en JSON via TCP vers Logstash. Il gère la reconnexion automatique si Logstash n'est pas disponible, et met les messages en tampon (max 100) pour ne rien perdre.

```
backend/src/logger/logstash.transport.ts

export class LogstashTransport extends TransportStream {
  private client: net.Socket | null = null;
  private connected = false;
  private buffer: string[] = []; // Tampon si Logstash indisponible

  private connect(): void {
    this.client = new net.Socket();
    this.client.connect(this.port, this.host, () => {
      this.connected = true;
      // Vide le tampon – envoie les logs mis en attente
      this.buffer.forEach(msg => this.client!.write(msg));
      this.buffer = [];
    });
    // Si connexion echouee, reessaie dans 5 secondes
    this.client.on('error', () => {
      this.connected = false;
      setTimeout(() => this.connect(), 5000);
    });
  }

  log(info: any, callback: () => void): void {
    // Construit le JSON enrichi avec les metadonnees SafeSchool
    const message = JSON.stringify({
      ...info,
      application: 'safeschool-backend',
      environment: process.env.NODE_ENV || 'development',
      timestamp: new Date().toISOString(),
    }) + '\n'; // json_lines : une ligne = un log

    if (this.connected && this.client) {
      this.client.write(message); // Envoi immediat
    } else {
      if (this.buffer.length < 100) { // Mise en tampon
        this.buffer.push(message);
      }
    }
  }

  callback();
}
```

```
}  
}
```

3.5 backend/src/logger/logger.service.ts — Le service central de logging

Ce service remplace tous les `console.log()` du projet. Il crée un logger Winston avec 2 transports : console (pour les logs Docker) et Logstash (pour Kibana). Il expose des méthodes spécialisées pour chaque type d'événement SafeSchool.

```
backend/src/logger/logger.service.ts
```

```
@Injectable()  
export class LoggerService implements NestLoggerService {  
  private readonly logger: winston.Logger;  
  
  constructor() {  
    const transports = [  
      new winston.transports.Console({ ... }), // Terminal Docker  
    ];  
  
    // Ajoute Logstash si LOGSTASH_HOST est defini dans .env  
    if (process.env.LOGSTASH_HOST) {  
      transports.push(new LogstashTransport({  
        host: process.env.LOGSTASH_HOST,  
        port: parseInt(process.env.LOGSTASH_PORT || '5044'),  
      }));  
    }  
  
    this.logger = winston.createLogger({ level: 'info', transports });  
  }  
  
  // Methode pour les requetes HTTP  
  http(data: { type: 'http_request'; method: string; url: string;  
    statusCode: number; responseTime: string;  
    userId?: string; userRole?: string }) {  
    this.logger.info(`${data.method} ${data.url} -> ${data.statusCode}`, data);  
  }  
  
  // Methode pour les evenements d'auth  
  auth(data: { type: 'auth_event'; action: string;  
    email: string; userId?: string; userRole?: string }) {  
    const level = data.action === 'login_failure' ? 'warn' : 'info';  
    this.logger.log(level, `[Auth] ${data.action}`, data);  
  }  
  
  // Methode pour les signalements  
  report(data: { type: 'report_event'; action: string;  
    caseNumber?: string; grade?: string; score?: number }) {
```

```
this.logger.info(`[Report] ${data.action} -- ${data.caseNumber}`, data);
}

// Methode pour le scoring IA
scoring(data: { type: 'scoring_event'; finalScore: number;
grade: string; urgency?: boolean; ... }) {
this.logger.info(`[Scoring] ${data.finalScore}/100 -> ${data.grade}`, data);
}
}
```

3.6 backend/src/logger/http-logger.middleware.ts — Toutes les requêtes HTTP

Ce middleware NestJS s'applique sur toutes les routes. Il log automatiquement chaque requête HTTP après que la réponse est envoyée, avec le temps de traitement, le code de statut et l'identité de l'utilisateur.

```
backend/src/logger/http-logger.middleware.ts

@Injectable()
export class HttpLoggerMiddleware implements NestMiddleware {
  constructor(private readonly logger: LoggerService) {}

  use(req: Request, res: Response, next: NextFunction): void {
    const { method, originalUrl, ip } = req;
    const start = Date.now();

    // S'exécute APRES que la réponse est envoyée au client
    res.on('finish', () => {
      const responseTime = Date.now() - start;
      const user = (req as any).user; // Injecte par Passport JWT

      this.logger.http({
        type: 'http_request',
        method,
        url: originalUrl,
        statusCode: res.statusCode,
        responseTime: `${responseTime}ms`,
        userId: user?.id || 'anonymous',
        userRole: user?.role || 'anonymous',
        ip: ip || 'unknown',
      });
    });

    next();
  }
}
```

3.7 backend/src/app.module.ts — Import du LoggerModule + middleware

Deux modifications dans app.module.ts : import du LoggerModule (marqué @Global()) pour être injectable partout sans réimport) et application du HttpLoggerMiddleware sur toutes les routes.

```
backend/src/app.module.ts

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
```

```
TypeOrmModule.forRoot({ ... }),
LoggerModule, // NOUVEAU - module global de logging
AuthModule,
UsersModule,
ReportsModule,
NotificationsModule,
StudentProfilesModule,
],
})
export class AppModule implements NestModule {
// NOUVEAU - applique le middleware HTTP sur TOUTES les routes
configure(consumer: MiddlewareConsumer) {
consumer.apply(HttpLoggerMiddleware).forRoutes('*');
}
}
```

3.8 Ajout des appels de logs dans les services métier

Dans auth.service.ts — 3 nouveaux logs

backend/src/auth/auth.service.ts

```
// Echec connexion (email inconnu)
this.logger.auth({ type: 'auth_event', action: 'login_failure',
  email, reason: 'email_not_found' });

// Echec connexion (mauvais mot de passe)
this.logger.auth({ type: 'auth_event', action: 'login_failure',
  email, userId: user.id, userRole: user.role, reason: 'wrong_password' });

// Connexion reussie
this.logger.auth({ type: 'auth_event', action: 'login_success',
  email, userId: user.id, userRole: user.role });
```

Dans reports.service.ts — 2 nouveaux logs

backend/src/reports/reports.service.ts

```
// Apres creation d'un signalement
this.logger.report({ type: 'report_event', action: 'created',
  reportId: savedReport.id, caseNumber, grade, score: finalScore,
  status: ReportStatus.PENDING, userId: student.id, userRole: student.role });

// Apres mise a jour d'un dossier
this.logger.report({ type: 'report_event', action: 'updated',
  reportId: id, caseNumber: report.caseNumber,
  grade: saved.grade, status: saved.status });
```

Dans scoring.service.ts — remplacement des console.log

backend/src/reports/scoring.service.ts

```
// AVANT (console.log basique)
console.log(`[Scoring] type:${typeScore} | base:${baseScore}`);

// APRES (log structure envoye vers Kibana)
this.logger.scoring({
  type: 'scoring_event',
  typeScore, frequencyScore, classScore,
  recidiveScore, suspectRoleScore, aiScore,
  finalScore, grade, aiReason, urgency,
});
```

4. Questions que l'évaluateur peut poser — avec les réponses

■ Question 1 : C'est quoi Elasticsearch et pourquoi ce n'est pas juste une base de données classique comme PostgreSQL ?

■ Elasticsearch est une base de données orientée recherche full-text basée sur Apache Lucene. La différence avec PostgreSQL c'est qu'Elasticsearch indexe chaque mot de chaque champ automatiquement, ce qui permet des recherches en millisecondes sur des millions de logs. PostgreSQL est optimisé pour les transactions et les relations entre données (signalements, utilisateurs...), pas pour la recherche dans des textes non structurés comme des logs. Dans SafeSchool on utilise les deux : PostgreSQL pour les données métier, Elasticsearch pour les logs.

■ Question 2 : Pourquoi passer par Logstash plutôt qu'envoyer directement de NestJS vers Elasticsearch ?

■ Logstash joue le rôle d'intermédiaire intelligent. Il reçoit les logs bruts de NestJS, les enrichit (ajoute des tags selon le type, parse les timestamps, met le level en majuscules) et les transmet à Elasticsearch dans le bon format. Si on envoyait directement depuis NestJS vers Elasticsearch, il faudrait gérer tout ça dans le code NestJS. Logstash sépare les responsabilités : NestJS se concentre sur la logique métier, Logstash sur le traitement des logs.

■ Question 3 : Comment NestJS envoie les logs vers Logstash ?

■ J'ai créé un transport TCP personnalisé (LogstashTransport) qui étend la classe TransportStream de Winston. Il ouvre une connexion TCP vers Logstash sur le port 5044 et envoie chaque log en JSON sur une ligne (format json_lines). Le transport gère la reconnexion automatique si Logstash n'est pas disponible, et met les messages en tampon (max 100) pendant la reconnexion pour ne rien perdre.

■ Question 4 : Que se passe-t-il si Logstash est en panne ? Les logs sont perdus ?

■ Non, grâce au système de tampon dans LogstashTransport. Quand Logstash est indisponible, les logs sont stockés dans un tableau en mémoire (buffer). Dès que la connexion est rétablie, le transport vide automatiquement ce tampon et envoie tous les logs en attente. Le tampon est limité à 100 messages pour éviter une consommation mémoire excessive. Les logs continuent aussi d'être affichés dans la console Docker même quand Logstash est down.

■ Question 5 : C'est quoi la politique ILM et comment tu as implémenté la rétention de 30 jours ?

■ ILM signifie Index Lifecycle Management. C'est un mécanisme d'Elasticsearch qui gère automatiquement le cycle de vie des index. J'ai configuré 3 phases : hot (7 jours, données actives et accessibles rapidement), warm (23 jours, données archivées, compressées pour économiser l'espace), et delete (à 30 jours, suppression automatique). Cette politique est créée au démarrage via un conteneur elk-setup qui exécute un curl vers l'API d'Elasticsearch. Le template d'index s'assure que tous les nouveaux index safeschool-logs-* héritent automatiquement de cette politique.

■ Question 6 : Pourquoi tu as utilisé un indice par jour (safeschool-logs-2026.04.15) plutôt qu'un seul index ?

■ Un index par jour facilite énormément la gestion du cycle de vie. Pour supprimer les logs de plus de 30 jours, il suffit de supprimer l'index du jour J-30, ce qui est instantané. Si on avait un seul index avec tous les logs, il faudrait faire une requête de suppression complexe sur chaque document. En plus, les index plus petits sont plus rapides à indexer et à chercher dans Kibana.

■ Question 7 : Quelle est la différence entre une note de type 'note' et une 'convocation' dans les logs ?

■ Dans SafeSchool, les notes de type 'note' sont internes à l'administration, invisibles par l'élève. Les convocations sont visibles par l'élève dans son dashboard. Dans les logs, on enregistre les deux avec le champ action: 'note_added' ou action: 'convocation_sent', ce qui permet dans Kibana de filtrer uniquement les convocations envoyées dans une période donnée, et de vérifier si elles ont bien été reçues.

■ Question 8 : Comment accéder à Kibana et qu'est-ce qu'on peut y voir pour SafeSchool ?

■ Kibana est accessible sur <http://localhost:5601> une fois Docker lancé. Dans Kibana on peut créer un Data View sur l'index safeschool-logs-* et explorer tous les logs. On peut voir en temps réel les connexions (réussies et échouées), les signalements créés avec leur grade et score, les détails du calcul de scoring IA, les temps de réponse de chaque endpoint, et créer des graphiques comme l'évolution du nombre de signalements par heure ou la répartition des grades. On peut aussi créer des alertes pour être notifié quand un log d'urgence arrive.

■ Question 9 : Est-ce que la sécurité d'Elasticsearch est configurée ?

■ Dans la configuration actuelle, `xpack.security.enabled` est à `false` pour simplifier le développement. En production, il faudrait activer `xpack.security`, configurer des certificats TLS pour le chiffrement des communications entre les composants, et créer des utilisateurs avec des rôles spécifiques (par exemple un utilisateur en lecture seule pour Kibana). Le sujet demande de sécuriser l'accès à tous les composants - c'est la prochaine étape : activer l'auth Elasticsearch et configurer Kibana avec un utilisateur/mot de passe.

■ Question 10 : Pourquoi le LoggerModule est marqué @Global() ?

■ Sans @Global(), il faudrait importer LoggerModule dans chaque module NestJS qui veut utiliser LoggerService : AuthModule, ReportsModule, UsersModule... Ce serait répétitif. Avec @Global(), NestJS rend LoggerService disponible dans toute l'application dès qu'il est importé une seule fois dans AppModule. C'est une bonne pratique pour les services transversaux comme le logging, la configuration ou la base de données.

5. Récapitulatif — Tous les fichiers du module ELK

Fichier	Statut	Rôle
docker-compose.yml	Modifié	Ajout des 4 services : elasticsearch, logstash, kibana, elk-setup
elk/logstash/pipeline/logstash.conf	Créé	Pipeline Logstash : input TCP 5044 → filter → output Elasticsearch
elk/logstash/config/logstash.yml	Créé	Configuration générale Logstash (port monitoring, xpack off)
elk/setup/ilm-policy.json	Créé	Politique de rétention : hot 7j → warm 23j → delete 30j
elk/setup/setup.sh	Créé	Script d'init : crée la politique ILM et le template d'index
backend/src/logger/logstash.transport.ts	Créé	Transport TCP Winston → Logstash avec reconnexion et tampon
backend/src/logger/logger.service.ts	Créé	Service central avec méthodes http(), auth(), report(), scoring()
backend/src/logger/http-logger.middleware.ts	Créé	Log automatique de toutes les requêtes HTTP
backend/src/logger/logger.module.ts	Créé	Module @Global() — injectable partout sans réimport
backend/src/main.ts	Modifié	Utilise LoggerService Winston comme logger NestJS principal
backend/src/app.module.ts	Modifié	Import LoggerModule + application HttpLoggerMiddleware
backend/src/auth/auth.service.ts	Modifié	Ajout logs : login_success, login_failure, register
backend/src/reports/reports.service.ts	Modifié	Ajout logs : report created, report updated
backend/src/reports/scoring.service.ts	Modifié	Remplacement console.log par logger.scoring() structuré
backend/package.json	Modifié	Ajout dépendances : winston, winston-transport
.env.example + .env	Modifié	Ajout : LOGSTASH_HOST, LOGSTASH_PORT, LOG_LEVEL

■ Pour démarrer : `docker compose up --build` — puis ouvrir `http://localhost:5601` pour Kibana. Les logs apparaissent en temps réel dès qu'un utilisateur se connecte ou soumet un signalement.